

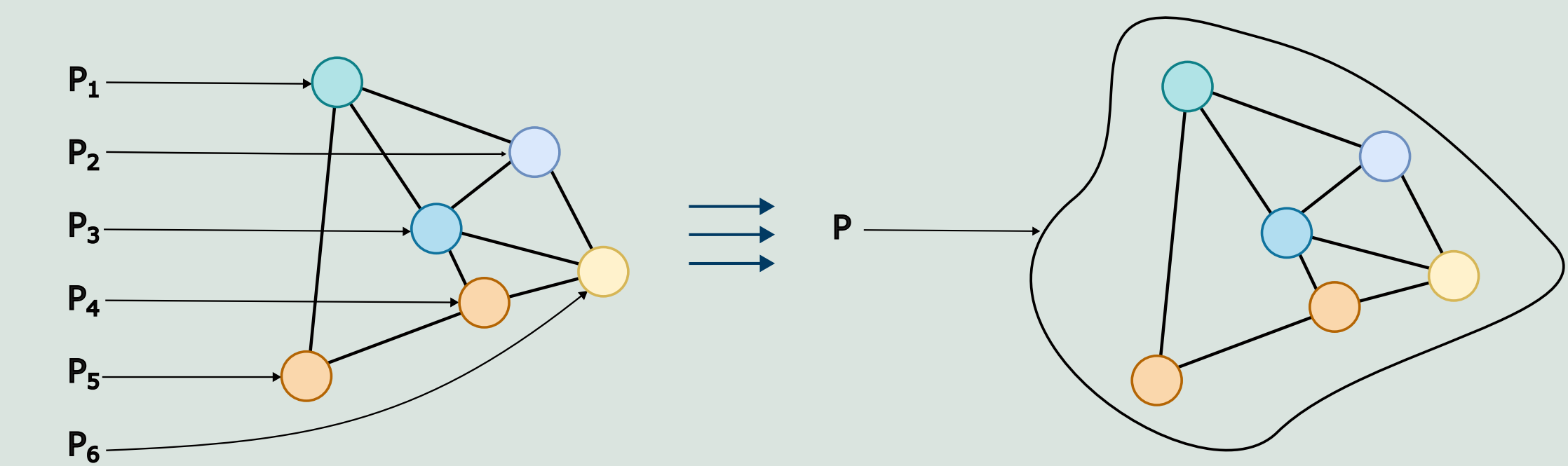
Towards Multiplatform Self-Organizing Aggregate Computing Systems



Angela Cortecchia
Supervisor: Prof. Danilo Pianini

Fellow researcher @GARR & (soon) PhD student @UniBo
angela.cortecchia@unibo.it

Aggregate Computing



Aggregate Computing (AC) is a macro-programming approach for **self-organizing systems**. It is based on the **Field Calculus**, which manipulates distributed data structures called **computational fields**.

AC defines the **collective behavior of heterogeneous devices**.

State of the art

FC

PP

```
field<double> f = nbr(CALL, 4.2);  
//manual alignment  
int n = nbr(CALL, 0, [&](field<int> a){  
    return min_hood(CALL, a);  
});
```

E

```
val y = minHood(nbr(id))  
val ids = nbr(id)  
// astonishing behavior  
minHood(ids) // -> returns local ID
```

P

```
let n = nbr(id)  
let y = mux (n > 0.5) { [1,2] } else { 0 }  
y.first() //possible error at runtime
```

Transparent alignment

Alignment is performed behind the scenes by a compiler plugin.

Reified fields

Direct field manipulation simplifies programming and reduces surprising behaviors.

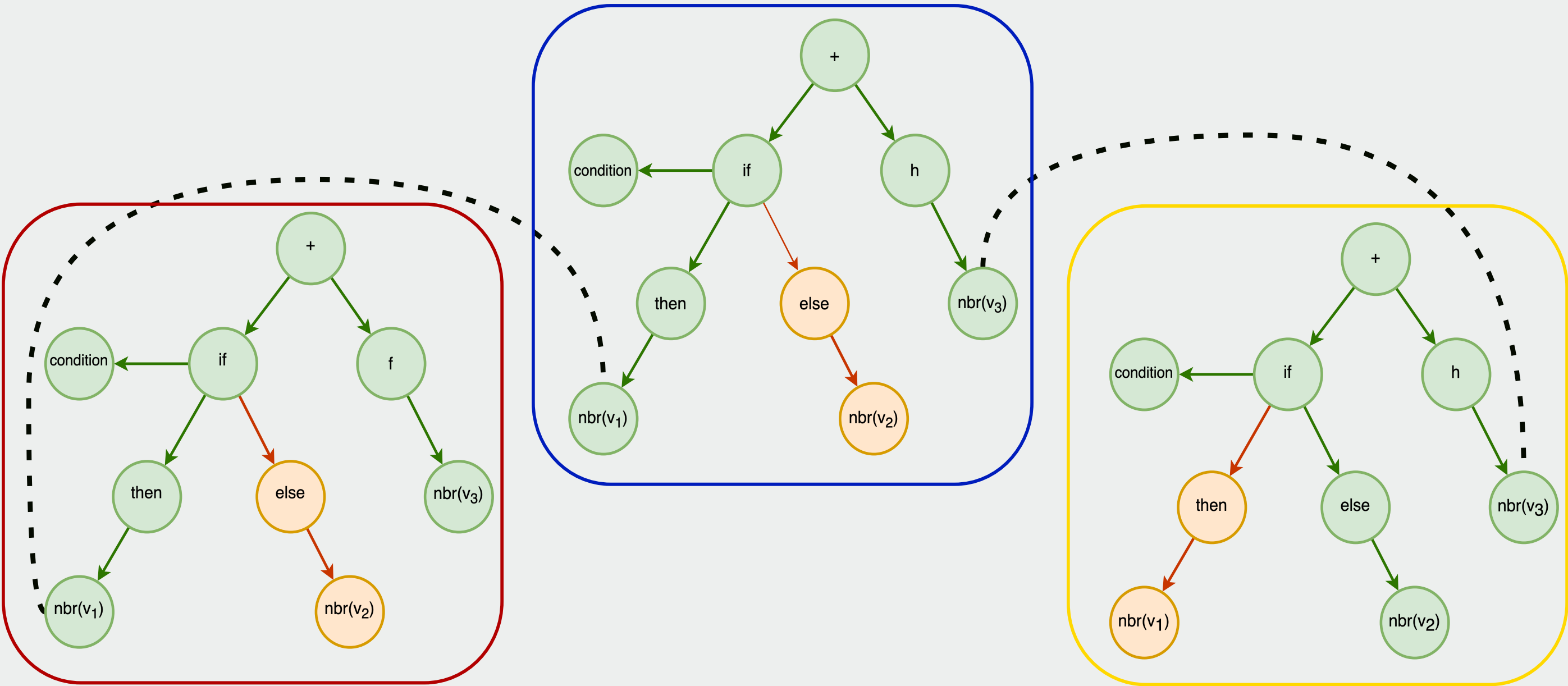
Static checking

Type checking at compile time can prevent unexpected errors at runtime.

```
fun Aggregate<Double>.x() {  
    val n = neighboring(localID)  
    n.map { listOf(it, 2.0, 3.0) }.first() // Field<Double>  
    // Typed fields prevent wrong calls at compilation time  
    n.map { if (it > 0.5) listOf(1, 2) else 0 } // Field<Any>  
    .first() // Error: receiver type mismatch -- CORRECT!  
}
```

Alignment (a.k.a. magic implicit communication)

In Aggregate Computing, **communication is implicit** (there is no "send message" or "receive message" primitive).
How? Thanks to "**alignment**".
Two (ore more) devices are "**aligned**" at some point in the program if they both have evaluated the **same abstract syntax tree node**.
Values of neighboring aligned devices are visibile when computing.



Idea

Implement alignment directly into a mainstream programming language **compiler**.
Pick a programming language that supports **multiple targets** and **internal domain specific languages (DSLs)**.
=> **Kotlin Multiplatform**

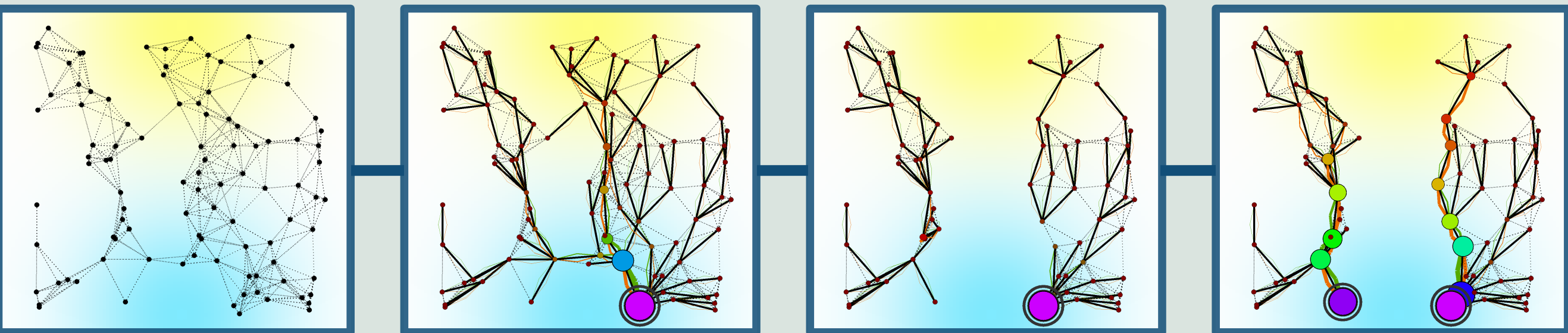
Collective: Aggregate Programming in Kotlin-MP with compiler-level alignment



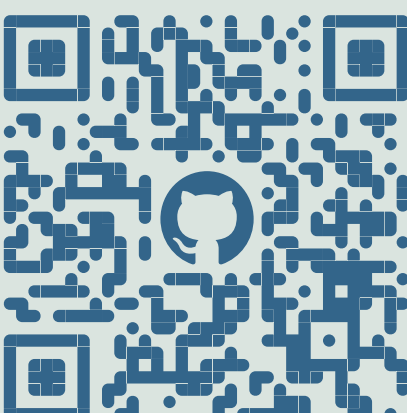
Language	DSL type	JVM	JS	Wearables	Reified Fields	Transparent Alignment	Automatic Complete Alignment
Proto	external	X	X	X	✓	✓	✓
Protelis	external	✓	X	X	✓	✓	✓
ScaFi	internal (Scala)	✓	✓	~	X	~	X
FCPP	internal (cpp)	X	X	~	✓	X	X
Collective	internal (Kotlin)	✓	✓	✓	✓	✓	✓

✓ Supported
X Not supported
~ Partially supported

Application Example



A generalization of the Vascular Morphogenesis Controller:
self-assembly and self-improvement of spatial structures.



Future Works

- ➡ Comprehensive **evaluation through simulations**.
- ➡ Execution on **different platforms**.
- ➡ Development of a **standard library**, with support for **reusable building blocks**.
- ➡ Towards a **collective operating systems** based on Aggregate Computing.

